Atividade Prática: Correção de Erros de Lógica em Programas C

📌 **Objetivo:** Desafiar os alunos de **técnico em Lógica de Programação e Desenvolvimento de Sistemas** a corrigir erros de lógica em **scripts C**, com foco em **estruturas de dados**, **entrada e saída de dados**, **enlace de repetição**, **condicionais**, **operadores aritméticos e lógicos**, **vetores**, **registros**, **listas**, **pilha** e **fila**. Eles deverão entender os **conceitos básicos** de programação, corrigir os erros e justificar as modificações realizadas.

📌 **Formato: Atividade Individual**
📌 **Duração: 2 horas**
📌 **Ferramentas: IDE de C** (como Dev C++, Code::Blocks, ou IDE online como replit.com)
📌 **Entrega:** Código corrigido em **.txt** (Bloco de Notas), com explicações sobre a correção de erros e justificação.

📖 Contextualização e Situação Problema:

Você foi chamado para corrigir uma série de **scripts C** usados em um **sistema de controle de estoque**, **sistema bancário**, **controle de atendimento ao cliente**, e **sistema de fila de tarefas**. Esses scripts estão com **erros de lógica** e precisam ser corrigidos para funcionar corretamente.

História do Mundo Real:

Um **sistema de loja online** está com problemas, pois alguns clientes não conseguem **finalizar o pedido** devido a erros no código, e o **sistema bancário** falha ao tentar **calcular os valores das transferências**. O **sistema de fila de clientes** não está **atualizando corretamente** as ordens de atendimento, causando longas esperas. Como programador, você precisa corrigir esses **erros lógicos** para melhorar a experiência do cliente e garantir que o sistema funcione adequadamente.

📋 Passo a Passo de Execução:

1. Preparação do Ambiente (10 minutos):

- Abra o **compilador de C** (Dev C++, Code::Blocks ou IDE online).
- Crie um **novo arquivo** de código em C e salve com o nome `corrigir_erro_logico.c`.

2. Erros Lógicos e Justificativas (10 Scripts)

Aqui estão **10 scripts com erros lógicos** que os alunos deverão corrigir. Cada um deles aborda um **erro lógico diferente** nos conceitos de **lógica de programação** e **estruturas de dados**.

Problema 1: Erro de Condição em Verificação de Paridade

Script com erro:

C/C++

```c
#include <stdio.h>

int main() {
    int numero;
    printf("Digite um numero: ");
    scanf("%d", &numero);

    if (numero % 2 == 1) {
        printf("O numero %d e par.\n", numero);
    } else {
        printf("O numero %d e impar.\n", numero);
    }

    return 0;
}
```

C/C++

Problema 2: Falha na Inicialização de um Vetor

Script com erro:

C/C++

```c
#include <stdio.h>
```

```c
int main() {
    int vetor[5];
    vetor[5] = 10;  // Erro: acesso fora do limite do vetor
    printf("%d\n", vetor[5]);
    return 0;
}
```

C/C++

Problema 3: Erro em Loop de Repetição

Script com erro:

C/C++
```c
#include <stdio.h>

int main() {
    int i;
    for (i = 0; i <= 10; i++) {  // Deve ser i < 10
        printf("%d ", i);
    }
    return 0;
}
```

C/C++

Problema 4: Falta de Validação de Entrada

Script com erro:

```C/C++
#include <stdio.h>

int main() {
    int idade;
    printf("Digite sua idade: ");
    scanf("%d", &idade);
    printf("Sua idade e: %d\n", idade);
    return 0;
}
```

```C/C++

```

Problema 5: Erro de Lógica em Busca Linear

Script com erro:

```C/C++
#include <stdio.h>

int main() {
    int vetor[] = {1, 2, 3, 4, 5};
    int n = 5, chave = 4;
    for (int i = 0; i < n; i++) {
        if (vetor[i] = chave) {  // Erro de atribuição em vez de
comparação
            printf("Chave encontrada!\n");
        }
    }
    return 0;
}
```

C/C++

Problema 6: Falha no Cálculo de Média com Vetor

Script com erro:

```
C/C++
#include <stdio.h>

int main() {
    int notas[] = {7, 8, 9, 10};
    int soma = 0, i;
    for (i = 0; i < 5; i++) {  // Erro: o índice está fora do
tamanho do vetor
        soma += notas[i];
    }
    printf("A media das notas e: %d\n", soma / 5);
    return 0;
}
```

```
C/C++
```

Problema 7: Erro em Pilha (Stack)

Script com erro:

```
C/C++
#include <stdio.h>
```

```
#define MAX 5

int pilha[MAX], topo = -1;

void push(int valor) {
    if (topo == MAX - 1) {
        printf("Pilha cheia!\n");
    } else {
        pilha[++topo] = valor;
    }
}

int main() {
    push(10);
    push(20);
    push(30);
    push(40);
    push(50);
    push(60);  // Erro: tentativa de empurrar para pilha cheia
    return 0;
}
```

Problema 8: Erro em Lista Encadeada (Inserção)

Script com erro:

C/C++

```
#include <stdio.h>

struct Node {
    int valor;
    struct Node* prox;
};

void inserir(struct Node* lista, int valor) {
```

```
    lista->valor = valor;  // Erro: não cria um novo nó
}

int main() {
    struct Node* lista = NULL;
    inserir(lista, 10);  // Erro: lista ainda não foi alocada
    return 0;
}
```

C/C++

Problema 9: Erro de Lógica em Fila (Queue)

Script com erro:

C/C++

```
#include <stdio.h>

#define MAX 5

int fila[MAX], frente = -1, tras = -1;

void enfileirar(int valor) {
    if (tras == MAX - 1) {
        printf("Fila cheia!\n");
    } else {
        fila[++tras] = valor;
    }
}

int main() {
    enfileirar(10);
```

```
    enfileirar(20);
    enfileirar(30);
    enfileirar(40);
    enfileirar(50);
    enfileirar(60);  // Erro: tentativa de enfileirar em fila cheia
    return 0;
}
```

Problema 10: Erro em Condicional para Validar Entrada de Dados

Script com erro:

C/C++

```c
#include <stdio.h>

int main() {
    int idade;
    printf("Digite sua idade: ");
    scanf("%d", &idade);

    if (idade > 18) {
        printf("Você é maior de idade.\n");
    } else if (idade >= 18) {
        printf("Você tem 18 anos ou mais.\n");
    } else {
        printf("Você é menor de idade.\n");
    }

    return 0;
}
```

C/C++

📄 Formato de Entrega:

1. **Correção do código:** O arquivo de código corrigido deve ser entregue no formato **.txt** (Bloco de Notas).

2. **Relatório Técnico (ABNT):** Um relatório explicando os erros identificados, como foram corrigidos e o raciocínio por trás de cada correção.

📋 Rubrica de Avaliação:

Critério	Descrição	Peso (%)
Correção do Código	O código foi corrigido corretamente para resolver os erros de lógica.	40%
Documentação Técnica (ABNT)	Relatório técnico bem escrito e seguindo as normas ABNT, explicando as correções.	30%
Boas Práticas de Programação	O código segue boas práticas (comentários, nomes significativos e indentação adequada).	20%
Testes e Resultados	O código foi testado adequadamente e os resultados estão de acordo com os requisitos.	10%

🎯 Resumo da Atividade:

Essa atividade desafia os alunos a **corrigir erros lógicos** em programas simples em **C** que envolvem conceitos importantes de **lógica de programação**, como **estrutura de dados**, **condicionais**, **loops**, **pilhas**, **listas**, **fila**, entre outros. As correções envolvem **análise crítica** e compreensão profunda dos **conceitos de programação**, além de proporcionar uma prática para o desenvolvimento de **boas práticas de programação** e **documentação técnica**.